Streamlit Project

Vincent Eung

Data Vizualisation - 09 octobre 2022

Hatim Chahdi

Contexte :

Au travers de ce travail nous avions à réaliser un projet avec streamlit.

Celui-ci devait comporter plusieurs librairies nous permettant de mettre en avant plusieurs visualisations avec plusieurs fichiers csv.

Pour ce faire j'ai pris l'initiative de travailler qu'avec deux fichiers csv ("full_2020.csv" et "full_2019.csv"). Puis trois les derniers jours, j'ai finalement implémenté le fichier csv de 2018.

Avec le fichier .py vous trouverez les différentes fonctions utilisées pour mener à bien mon travail.

Comment récupérer les fichiers csv ?

En copiant ce lien sur google, vous serez redirigé vers le github dans lequel vous aurez accès à tous les fichiers indispensables au travail.

https://drive.google.com/drive/folders/1R_9A9yPOzRQzMCyTDBEJms0u1ZCN7MbY

Mais quelle a été le but de ma recherche ?

Montrer avec les différents données la valeur du prix/m² par rapport aux indicateurs riches et variés proposés par le dataset.

Fonctions utilisées puis appeler dans le main :

1)

```
def load_data(path):
    df = pd.read_csv(path, warn_bad_lines=True, error_bad_lines=False,low_memory=False)
    return df.sample(frac = .10)
```

Cette fonction permet de lire tous les fichiers '.csv' du dossier. Ainsi après appel de la fonction et l'utilisation de ".sample" afficher aléatoirement les lignes et colonnes du dataset.

2)

```
def cleanning(df):
    df.drop(labels = ['adresse_suffixe','adresse_numero','ancien_code_commune','ancien_nom_commune',
                      'ancien_id_parcelle','numero_volume', 'lot1_numero',
                      'lot1_surface_carrez','lot2_surface_carrez','lot2_numero','lot3_numero',
                      'lot3_surface_carrez','lot4_numero','lot4_surface_carrez',
                     'lot5_numero','lot5_surface_carrez'],axis=1,inplace=True)
    df.dropna(subset=['valeur_fonciere','surface_terrain','longitude','latitude','code_commune',
     'nom_commune','code_type_local','type_local','surface_reelle_bati','nombre_pieces_principales',
     'nature_culture_speciale','code_nature_culture_speciale'],axis=0, inplace=True)
    df['Prix/m2'] = df['valeur_fonciere']/df['surface_terrain']
    return df
```

Cette fonction permet au programme après appel dans le main d'enlever les colonnes dites 'inutiles' à mon travail. En d'autres termes les éléments dispensables ou voire gênante pour mon travail. Par le biais de cette j'en ai profité pour ajouter une nouvelle colonne pour calculer le prix au mètre carré qui me sera utile tout le long du projet.

3)

```
def bar_prix_metre_carre(df):
    df=df.groupby(['code_commune'])['Prix/m2'].mean()
    st.bar_chart(df)
```

Cette fonction est utilisée à partir de la librairie de streamlit. Elle permet d'afficher le prix/m² pour chaque commune généré aléatoirement à partir du dataset.

4)

```
def prix_date_mutation(df):
    df = px.bar(df, x='date_mutation', y='Prix/m2')
    st.plotly_chart(df)
```

Cette fonction a été programmé avec la librairie « .plotly ». Permettant ainsi au programme d'afficher le prix/m² par rapport au différentes dates de mutation du dataset.

5)

```
def hist_surface_local(df):
    df = px.histogram(df, y='surface_reelle_bati', x='type_local')
    st.plotly_chart(df)
```

Cette fonction a été programmé avec la librairie « .plotly ». Elle permet de savoir quelle genre de local nous retrouvons selon les différents terrains.

6)

```
def prix_surface(df):
    c = alt.Chart(df).mark_bar().encode(
    x=alt.X('Prix/m2',bin=alt.Bin(maxbins=5)),
    y='surface_reelle_bati')
    st.altair_chart(c,use_container_width=True)
```

Cette fonction a été programmé avec la librairie « .altair ».. En complément de la fonction précédente, elle permet de savoir le prix/m² nous retrouvons selon les différents terrains.

7)

```
def prix_departement(df):
    df = px.pie(df, values='Prix/m2', names='code_departement')
    df.update_traces(textposition='inside', textinfo='percent+label')
    st.plotly_chart(df)
```

Cette fonction a été programmé avec la librairie « .plotly ». Elle permet à l'utilisateur de connaître le prix/m² selon les différents département.

8)

```
def prix_nature_culture(df):
    plot = plt.figure(figsize=(25,15))
    sns.barplot(data=df, x="nature_culture", y="Prix/m2")
    st.pyplot(plot)
```

Cette fonction permet à l'utilisateur de connaître le prix/m² pour les jardins, les terrains d'eau ou même les terrains d'agrément. L'utilisation avec la librairie « matplotlib.pyplot » a été utilisé pour afficher ce graphe.

9)

```
def prix_local(df):
    c = alt.Chart(df).mark_circle().encode(x=('type_local'),y='Prix/m2').interactive()
    st.altair_chart(c,use_container_width=True)
```

Cette fonction a été programmé avec la librairie « altair ». Elle permet à l'utilisateur de connaître le prix/m² selon les différents type de logement du dataset.

10)

```
def prix_code_nature_culture(df):
    fig =px.scatter(df, x="code_nature_culture", y="Prix/m2", color='nature_mutation')
    fig.update_layout(width = 800)
    st.plotly_chart(fig)
```

Cette fonction a été programmé avec la librairie « .plotly ». Elle permet à l'utilisateur de connaître le prix/m² selon les différents nature du logement. C'est-à-dire qu'elle soit à vendre, vente, échange ou adjudication.

11)

```
def map(df):
    df = df[['code_departement', 'nature_culture', 'longitude','latitude']]
    df = df[df['code_departement'].str.contains('94')]
    df = df[df['nature_culture']==('jardins')]
    coordonnees=df[['longitude', 'latitude']]
    st.dataframe(df)
    return coordonnees
```

Cette fonction permet à streamlit d'afficher un map du monde. Mais la fonction va nous permettre de nous concentrer sur une partie. J'ai alors pris l'initiative de travailler sur le département du « 94 ». La fonction va ensuite chercher tous les jardins du dataset de la colonne « nature_culture » pour les afficher par la suite. En complément, je me suis permis de rajouter un tableau qui met en avant les prélèvement de données faits par le programme lui-même.

12)

```python
def main():

    st.title('Bienvenue sur la page accueil ! ')
    st.header('Data Vizualtion - 2022')
    st.write('👈Au sein de la sidebar, vous serez apte à afficher toute la visualisation que vous désirez selon une année de votre choix.👈')

    with st.sidebar:
        st.header('Programmé par :')
        st.write('Vincent Eung🧑')
        st.write('BIA 1💻')
        st.write('Student Code ✅ : 20190605')

        image = Image.open('pdp.jpg')
        st.image(image)

        ############ 3 checkbocks ############

        st.header('Cocher une ou plusieurs cases pour afficher lannée souhaitée :')
        choice = st.checkbox("2020")
        choice2 = st.checkbox("2019")
        choice3 = st.checkbox("2018")

    select = st.radio("Si vous désirez afficher le tableau, sélectionner un des boutons suivants :",('Reset','2020','2019','2018'))

    if select == '2020':    ...
    elif select == '2019':    ...
    elif select == '2018':    ...

    if choice:
        df_2020 = load_data("full_2020.csv")
        cleanning(df_2020)
```

Dans la fonction main, je me suis permis d'ajouter une sidebar pour rendre le projet plus dynamique. Dans celle-ci on retrouve plusieurs choses. La première, des informations me concernant. La seconde, les différentes « checkbox » demandées. Chaque d'entres elle permettent à l'utilisateur d'afficher et de retirer toutes les visualisations générées par les fonctions. On retrouve également des « radio button» permettant à l'utilisateur d'afficher à sa guise le dataset nettoyé.

Pour cette dernière fonctionnalité, l'utilisation de fonction « if » a été nécessaire pour permettre l'affichage des différentes datasets des différentes années.

Problèmes rencontrés :

1) L'utilisation de « st.cache » me bloquait l'accès à plusieurs fonctionnalités. A chaque reload de la page, mon dataset n'arrivait plus à charger correctement. Une erreur me confirmant belle et bien la suppression des colonnes mais ne pouvait elle-même affiché. J'ai alors dû faire le choix de le mettre en commentaire.
2) Contre toute attente la suppression des colonnes a été plus long que ce que je ne pouvais imaginer. Car oui l'utilisation de « labels » à la place de « columns » pour la suppression des NAN et des colonnes indésirables a été plus efficace.
3) Lors de ce projet je n'ai pu afficher de slider, un timer et des graphes fonctionnels avec la librairie « bokeh » :
    - Bokeh : l'utilisation de cette librairie est pour la plus compliquée à utiliser. Malgré d'innombrables efforts je n'ai pu afficher correctement mes graphes.
    - Slider : Par manque je n'ai pas trouvé la manière dont je voulais l'implémenter dans mon projet. Or je pu l'implémenter mais pas correctement. J'ai alors pris la décision de ne pas le mettre malgré le fait de l'avoir affiché.
    - Timer/Decorator : Même problème, par manque de temps je n'ai pas eu le temps de rechercher le moyen de le faire avec tous les péripéties traversées lors de ce projet.
    - Mis à part ces problèmes, en voulant trop nettoyé le dataset je me suis retrouvé à ne plus avoir certaines communes comme « Paris ».